

Introduction to Information Technology Software

Võ Thiện Trí Nhân - 25127113

October 25 2025



Figure 1: Computer - Source: [Unsplash](#)

Contents

1	Introduction	3
2	Programming Languages - Compilers	3
2.1	Programming Languages	3
2.2	Compilers	4
3	Code editor - Integrated Development Environment	5
3.1	Integrated Development Environment . . .	5
3.2	Code editor	6
4	Different levels of Programming Languages	7
4.1	Low-level Programming Languages	7
4.2	Mid-level Programming Languages	8
4.3	High-level Programming Languages	8
4.4	Domain-specific Programming Languages .	9
5	Programming Language Characterize Criteria	9

1 Introduction

Greetings, my name is Võ Thiện Trí Nhân - Student ID: 25127113. I had written this report to recap my knowledge on

- Programming Languages
- Compilers
- Code editors
- Integrated Development Environments

During my research on these concepts, I discovered even more fascinating information in Information Technology. I hope this report helps you understand these fundamental concepts and apply them in real-life scenarios.

Disclaimer

This report was authored by Võ Thiện Trí Nhân using Overleaf ($\text{\LaTeX}$). I also utilized Copilot to assist me with $\text{\LaTeX}$ commands and to elaborate on the concepts.

2 Programming Languages - Compilers

- A computer can only understand “Machine Language”, which contains zeros and ones. Humans can interpret Binary.
 - However, we are used to the Decimal number system, which makes programming directly to computers complicated and slow.
- In the past, engineers had to turn switches on and off to program something into the computer.
 - This process is complicated, time-consuming, and can only be handled by specialized engineers.

Therefore, programming languages and compilers are created to handle these jobs

2.1 Programming Languages

Look at this example:

- A chef wants to write an instruction on how to make a cake. Besides the ingredients and tools, he/she needs **a means of communication** to note the instructions.
 - He/she can use “languages” (Vietnamese, English, and Chinese) to list down the steps.
 - In those “languages”, there are keywords he/she can use to emphasize the jobs that need to be completed to bake a cake.

- Languages need to be stored on something. It can be a paper (writing) or a recorder (speaking).
- Programming Languages do the same job as the “languages” in the example.
 - They provide the programmers with syntax and commands (which are keywords in the example) to write the source code (the instructions).
 - The programmer has to learn these syntaxes and commands (similar to how the person has to learn the vocabulary to write the steps).
- Programming Languages need to be stored on a “storage”. Programmers usually use a code editor to modify and a file to store the source code.

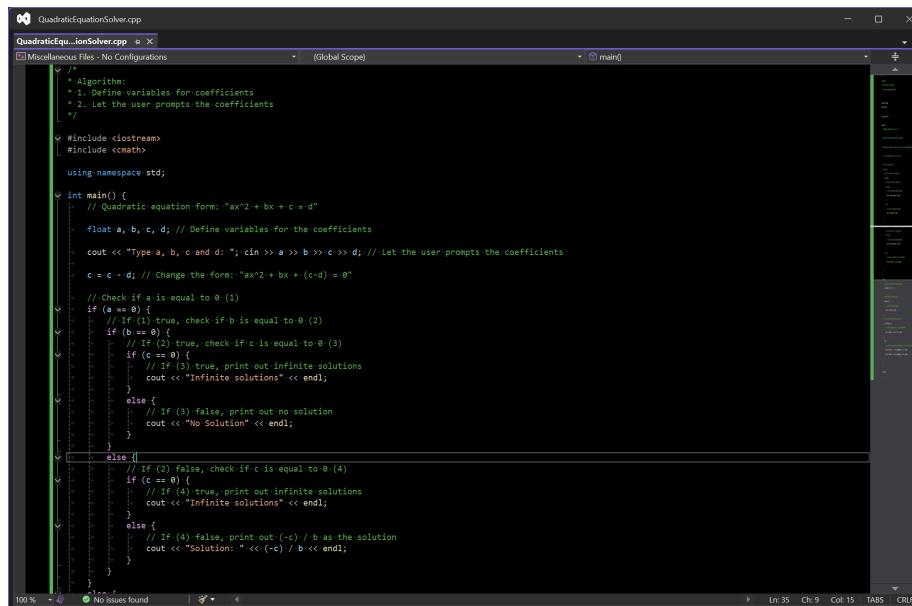
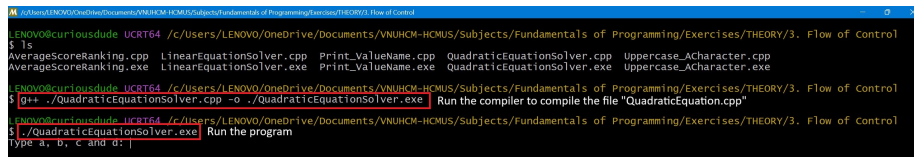


Figure 2: Programming Language - C++

2.2 Compilers

- If the chef wants to give this instruction to a foreign chef, he/she needs to translate that paper from one language to another.
 - Therefore, he/she has to give this paper to a middleman who knows the foreign chef’s language to translate.
- After the paper is translated, it will be passed to the foreign chef. Finally, the foreign chef can comprehend the instructions and remake the cake based on that.

- Like we have said before, the chef is a programmer, the language he/she uses to write down steps is a programming language. In this example, the middleman is a compiler, and the foreign chef is a computer.
 - The compiler acts as a translator, translating the source code written in human languages to machine language and making a new instruction so that the computer will act based on that and take the steps.
- Compilers are usually executed on the Terminal. Some can be integrated into the Integrated Development Environment (IDE).



```
LENOVO@curiousdude: UCR164 /c/Users/LENOVO/OneDrive/Documents/VNUHCM-HCMUS/Subjects/Fundamentals of Programming/Exercises/THEORY/3. Flow of Control
$ ls
AverageScoreRanking.cpp  LinearEquationSolver.cpp  Print_ValueName.cpp  QuadraticEquationSolver.cpp  Uppercase_ACharacter.cpp
AverageScoreRanking.exe  LinearEquationSolver.exe  Print_ValueName.exe  QuadraticEquationSolver.exe  Uppercase_ACharacter.exe
LENOVO@curiousdude: UCR164 /c/Users/LENOVO/OneDrive/Documents/VNUHCM-HCMUS/Subjects/Fundamentals of Programming/Exercises/THEORY/3. Flow of Control
$ g++ ./QuadraticEquationSolver.cpp -o ./QuadraticEquationSolver.exe Run the compiler to compile the file "QuadraticEquation.cpp"
LENOVO@curiousdude: UCR164 /c/Users/LENOVO/OneDrive/Documents/VNUHCM-HCMUS/Subjects/Fundamentals of Programming/Exercises/THEORY/3. Flow of Control
$ ./QuadraticEquationSolver.exe Run the program
type a, b, c and d:
```

Figure 3: Compiler - mingw-w64

3 Code editor - Integrated Development Environment

3.1 Integrated Development Environment

- A heavyweight software includes:
 - Code editor: Let the programmer modify the source code.
 - Compiler/Interpreter: Translate the source code from human languages to the computer language
 - Debugger: If the programmer encounters a bug and he/she doesn't know where the bug comes from, the debugger lets the programmer run the source code line by line to see how the computer works behind the scenes to figure out the error.
 - And other tools integrated into the software.
- IDEs are suitable in enterprises for massive projects because of their capabilities. However, they are heavy and can cause your RAM to be full quickly.

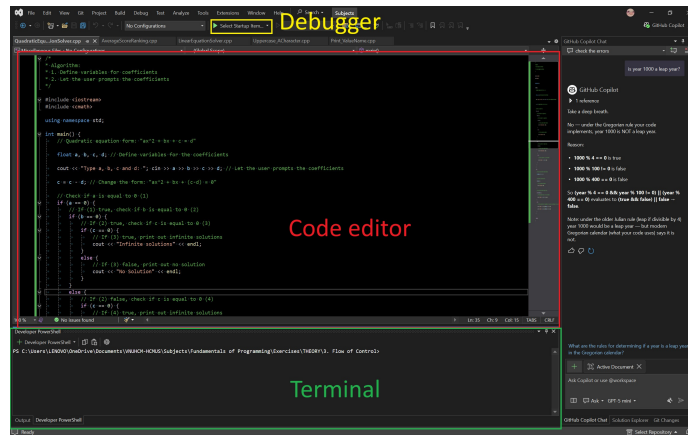


Figure 4: IDE - Visual Studio

3.2 Code editor

- A lightweight software includes only a code editor. If the programmer wants more tools in the code editor, they can install those through extensions.
- These code editors are suitable for small personal projects. They are also lightweight, which cannot fill up your RAM quickly.

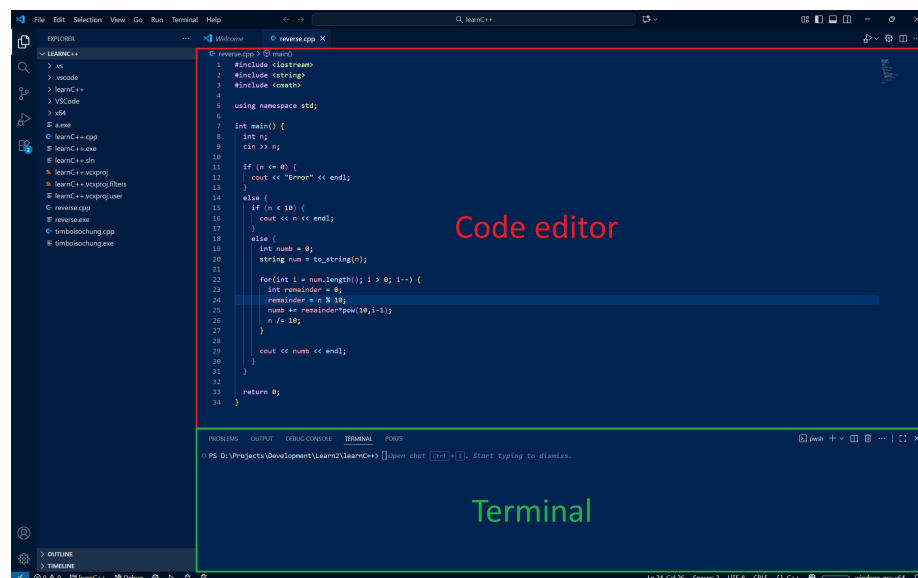


Figure 5: Code editor - Visual Studio Code

4 Different levels of Programming Languages

4.1 Low-level Programming Languages

- These PLs are closest to machine code and provide minimal abstraction from the hardware.
- They are easy for computers to work with, but hard for humans to comprehend and program.

```
.model small
.stack 100h
.data
a db ?
b db ?
c db ?
total db ?

.code
main:
mov ax, @data
mov ds, ax

; Prompt and read 'a'
mov ah, 1
int 21h
sub al, '0'
mov a, al

; Prompt and read 'b'
mov ah, 1
int 21h
sub al, '0'
mov b, al

; Prompt and read 'c'
mov ah, 1
int 21h
sub al, '0'
mov c, al

; Calculate total = a + b + c
mov al, a
add al, b
add al, c
mov total, al
```

```
; Convert total to ASCII and print
add al, '0'
mov dl, al
mov ah, 2
int 21h

; Exit
mov ah, 4Ch
int 21h
end main
```

Assembly

4.2 Mid-level Programming Languages

- These PLs are still close to machine language, but the syntaxes are familiar to human languages.

```
#include <iostream>
using namespace std;
int main() {
    int a, b, c;
    cout << "Type three numbers: "; cin >> a >> b >> c;
    cout << "Total: " << a + b + c << endl;
    return 0;
}
```

C++

4.3 High-level Programming Languages

- These PLs are close to human languages. They can comprehend the syntax much more easily.
 - However, the compiler or interpreter needs to take more time to translate these syntaxes.

```
import random as rm
def main():
    ran = int(input("Type a number: ").strip())
    randnum = rm.randrange(1, ran)
    while True:
        guess = int(input("Type a number: ").strip())
        if (guess <= 0):
            print("ERROR")
```



```
        continue
    else:
        if (guess == randnum):
            print("Congratulations")
            break
        else:
            print("Guess again: ")
            continue
```

Python

4.4 Domain-specific Programming Languages

- Languages provide greater abstraction, which is used for specialized tasks like data processing and Artificial Intelligence.

```
-- Create a table
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Age INT,
    Major VARCHAR(100)
);

-- Insert data into the table
INSERT INTO Students (StudentID, FirstName, LastName, Age, Major)
VALUES
(1, 'Huy', 'Nguyen', 18, 'Cybersecurity'),
(2, 'Nhan', 'Vo', 18, 'Artificial Intelligence'),
(3, 'Bao', 'Le', 18, 'Computer Science');

-- Query: Select all students majoring in Computer Science
SELECT * FROM Students
WHERE Major = 'Computer Science';
```

Structured Query Language

5 Programming Language Characterize Criteria

1. Proximity to Human Language

- Low-level Programming Languages have syntaxes closer to the machine language.
- Example: Assembly, C++

2. Ease of use

- The higher-level Programming Languages are designed to be easier to read, understand, and modify.
- You can check back the code [\[4.2\]](#) [\[4.3\]](#)
 - It is hard for the programmer to comprehend the meaning of a certain syntax in C++.
 - * Example: cout
 - It is much easier for the programmer to comprehend the meaning of a certain syntax in Python.
 - * Example: print

3. Efficiency of execution

- The low-level Programming Languages take less time to run.
 - The syntaxes are much closer to the computer language.
 - Most low-level programming languages use a compiler to translate the source code.
 - * The compiler's translation speed may be slow, but once it is completed, the execution speed is fast.
- The high-level Programming Languages take more time to run.
 - The syntaxes are much closer to human languages, not computer ones.
 - Most high-level programming languages use an interpreter to translate the source code
 - * The interpreter's translation speed may be fast, but the process has to be run over and over when executing.